Лабораторная работа №4. Наследование. Часть 1. Базовые аспекты

Наследование в Java — это механизм, позволяющий одному классу (подклассу, дочернему классу) наследовать поля и методы другого класса (суперкласса, родительского класса). Это основа принципа повторного использования кода и создания иерархий классов. Рассмотрим ключевые аспекты с примерами.

Ключевое слово extends

Ключевое слово extends (расширять) указывает, что дочерний класс наследуется от родительского и перенимает все его методы и переменные (кроме конструкторов), если они не помечены модификаторами доступа **private**. Далее идет пример, все классы объявлены в одном файле с именем *Main* :

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal { // Dog наследует Animal
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat();  // Метод из Animal
        dog.bark(); // Метод из Dog
    }
}
```

Вывод:

```
Animal is eating
Dog is barking
```

Доступ к членам класса

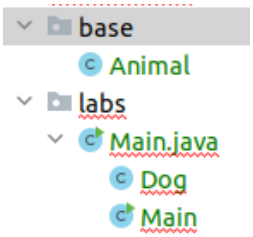
- Подкласс наследует и имеет доступ с членам родительского класса, если его поля и методы имеют модификаторы доступа **public** и **protected**.
- **private** члены класса недоступны напрямую, но можно использовать через публичные методы суперкласса, например геттеры и сеттеры.
- **protected** члены класса доступны в подклассе и всем другим классам в области пакета.
  T.e. если мы поместим класс Animal в другую папку проекта, например, base, то метод eat будет виден всем классам наследникам, включая Dog, но не виден остальным. Пример:

Main.java | Animal.java

```java
package org.example.base;

public class Animal {
    protected void eat() {
        System.out.println("Animal is eating");
    }
}
```

Классы разделены по папкам:



Результат разделения:

```java
class Dog extends Animal { // Dog наследует Animal
    void bark() {
        System.out.println("Dog is barking");
        eat(); // Метод виден здесь
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat();  // Но не виден здесь
        dog.bark(); // Метод из Dog
    }
}
```

Ключевое слово super

Конструкторы не наследуются, но можно вызвать конструктор суперкласса через *super*(). Например, создадим два класса: Animal и Dog. Dog будет наследовать Animal. В конструкторе класса Dog вызовем конструктор класса Animal через ключевое слово *super* и передадим имя:

```
class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

class Dog extends Animal {
    private String breed;

    public Dog(String name, String breed) {
        super(name); // Вызов конструктора Animal
        this.breed = breed;
    }

    public void printInfo() {
        System.out.println("Name: " + super.getName() + ", Breed: " + breed);
    }
}
```

В main создадим объект класса Dog и вызовем метод *printInfo:*

```

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog( name: "Doggy",  breed: "bulldog");
        dog.printInfo();
    }
}
```

Вывод:

```
Name: Doggy, Breed: bulldog
```

Важно иметь ввиду, если у суперкласса нет пустого конструктора мы обязаны вызвать его конструктор в дочернем классе иначе будет ошибка:

```
18      public Dog(String name, String breed) {
19          this.breed = breed;
20      }
21
```

```
'home/vadim/IdeaProjects/MSVtest/src/main/java/org/example/labs/Main.java:18:43
java: constructor Animal in class org.example.labs.Animal cannot be applied to given types;
  required: java.lang.String
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

Причина: фактические и формальные списки аргументов различаются по длине.

Решение: создание пустого конструктора в родительском классе

```
3  class Animal {
4      private String name;
5
6      public Animal(String name) {
7          this.name = name;
8      }
9
10     public Animal() {} // <------
11
12     public String getName() {
13         return name;
14     }
15 }
```

Также ключевое слово используется для вызова методов суперкласса, это будет рассмотрено далее.

Переопределение методов (Method Overriding)

Создадим новый класс - Cat и новый метод в классе Animal - *makeSound.* Переопределим новый метод в классах Cat и Dog для этого создадим точно такую же сигнатуру методов, что и в родительском классе, а также добавим аннотацию **@Override** (см. рис. ниже). Аннотация - это механизм с особым синтаксисом, который хранит какую-либо дополнительную информацию об объекте.

```java
class Animal {
    protected void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Cat extends Animal {
    @Override
    protected void makeSound() {
        System.out.println("Cat: Meow!");
    }
}

class Dog extends Animal {
    @Override
    protected void makeSound() {
        System.out.println("Dog: Woof!");
    }
}
```

Далее создадим объекты этих классов и вызовем метод *makeSound*:

```java
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();

        Cat cat = new Cat();
        cat.makeSound();
    }
}
```

Вывод:

```
Dog: Woof!
Cat: Meow!
```

Важно при переопределении методов и полей в дочерних классах, чтобы они имели одинаковую сигнатуру (имя, возвращаемый тип, количество и тип аргументов) и уровень доступа не должен быть более строгим, чем у родительского класса.

Также мы можем вызывать методы родительского класса с помощью *super*:

```
9   class Cat extends Animal {
10      @Override
11      protected void makeSound() {
12          super.makeSound();
13          System.out.println("Cat: Meow!");
14      }
15  }
```

Вывод:

```
Dog: Woof!
Animal makes a sound
Cat: Meow!
```

Часть 2. Полиморфизм

В прошлой части были рассмотрены базовые принципы наследования: ключевое слово **extends**, ключевое слово **super**, переопределение методов, модификатор доступа **protected**. В этой части мы рассмотрим полиморфизм, абстрактный класс и нюансы наследования.

Полиморфизм

Полиморфизм в Java — это одна из ключевых концепций объектно-ориентированного программирования (ООП), которая позволяет объектам одного типа вести себя по-разному в зависимости от контекста. Дословно "полиморфизм" означает "много форм".

Вернемся к примеру из прошлой части:

```java
class Animal {
    protected void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Cat extends Animal {
    @Override
    protected void makeSound() {
        super.makeSound();
        System.out.println("Cat: Meow!");
    }
}

class Dog extends Animal {
    @Override
    protected void makeSound() {
        System.out.println("Dog: Woof!");
    }
}
```

На самом деле мы можем вместо явного указания какой класс к какому относится, написать их общего предка, вместо этого кода:

```java
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();

        Cat cat = new Cat();
        cat.makeSound();
    }
}
```

Написать следующий:

```java
public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.makeSound();

        Animal cat = new Cat();
        cat.makeSound();
    }
}
```

Результат будет таким же:

```
Dog: Woof!
Animal makes a sound
Cat: Meow!
```

Это может быть полезно, если мы не знаем какой именно тип у нас будет в методе, но мы уверены, что есть общий класс, в который он включен и работать с ним как с общим объектом. Например, создадим поле *name* в классе Animal и статический метод *printName*, который в качестве аргумента принимает объект типа Animal, в теле методы напишем, что он выводит в консоль имя животного:

```java
class Animal {

    private String name;

    public Animal(String name) {
        this.name = name;
    }

    protected void makeSound() {
        System.out.println("Animal makes a sound");
    }

    public static void printName(Animal animal) {
        System.out.println(animal.name);
    }
}
```

Далее вызовем конструктор суперкласса в дочерних:

```java
class Cat extends Animal {

    public Cat(String name) {
        super(name);
    }

    @Override
    protected void makeSound() {
        super.makeSound();
        System.out.println("Cat: Meow!");
    }
}
```

В классе Dog аналогично.

Далее пользуясь тем, что статические методы принадлежат классам, а не их объектам напишем следующий код:

```
41  public class Main {
42      public static void main(String[] args) {
43          Animal dog = new Dog( name: "Doggy");
44          Animal cat = new Cat( name: "Vasay");
45
46          Animal.printName(dog);
47          Animal.printName(cat);
48      }
49  }
```

Результат:

```
Doggy
Vasay
```

Также мы можем определить тип объекта, переданного в метод через ключевое слово **instanceof**:

```
public static void printName(Animal animal) {
    if (animal instanceof Dog) {
        System.out.println(animal.name + "good boy");
    } else {
        System.out.println(animal.name);
    }
}
```

Результат:

```
Doggy good boy
Vasay
```

Класс Object

Класс Object является общим для всех классов в Java - от него все классы неявно наследуются. У класса Object есть 11 публичных методов:

- **protected Object clone()** - создает новый объект, не отличающийся от клонируемого.
- **public boolean equals(Object obj)** - определяет, равен ли один объект другому.

- **protected void finalize()** - вызывается перед удалением неиспользуемого объекта.
- **public final Class<?> getClass()** - получает класс объекта во время выполнения.
- **public int hashCode()** - возвращает хэш-код, связанный с вызывающим объектом.
- **public final void notify()** - возобновляет исполнение потока, ожидающего вызывающего объекта.
- **public final void notifyAll()** - возобновляет исполнение всех потоков, ожидающих вызывающего объекта.
- **public String toString()** - возвращает символьную строку, описывающую объект.
- **public final void wait()** - ожидает другого потока исполнения.
- **public final void wait(long timeout)** - ожидает другого потока исполнения.
- **public final void wait(long timeout, int nanos)** - ожидает другого потока исполнения

В этом курсе мы так или иначе коснемся только 4-х методов: **equals, getClass, toString** и **toString.** Например, метод *toString,* по-умолчанию, если вызвать у любого класса он вернет следующее, нечто наподобии: org.example.labs.Dog@28a418fc

```
Animal dog = new Dog( name: "Doggy");
System.out.println(dog.toString());
```

Но нам нужно, чтобы он возвращал, что-то осмысленное, например, все значения переменных класса. Для этого мы должны его переопределить:

```
15      @Override
16      public String toString() {
17          return "Animal{" +
18                  "name='" + name + '\'' +
19                  '}';
20      }
21
```

Результат:

```
Animal{name='Doggy'}
```

Ключевые слова final и static в наследовании

Ключевое слово **final** в Java используется для ограничения изменений в коде. Оно может применяться к переменным, методам, классам и параметрам методов, обеспечивая безопасность, неизменность и контроль над поведением программы.

Final для переменных:

- Значение нельзя изменить после инициализации.
- Примитивные типы: значение фиксировано.
- Ссылочные типы: ссылка фиксирована (но сам объект может изменяться, если его поля не final).

Примитивы:

```
final int x = 10;
x = 20; // Ошибка компиляции: cannot assign a value to final variable
```

Ссылочные типы:

```
final Animal dog = new Dog( name: "Doggy");
dog.setName("Bobik"); // Можно изменять внутренние переменные
dog = new Dog( name: "Bob"); // Однако, менять сылку нельзя
```

Final для методов:

- Нельзя переопределить в подклассах.
- Используется, чтобы зафиксировать реализацию метода.

Метод *makeSound* нельзя переопределить:

```
3   class Animal {
4
5       public final void makeSound() {
6           System.out.println("Animal makes a sound");
7       }
8   }
9
10  class Cat extends Animal {
11      
12      @Override
13      public void makeSound() {
14          super.makeSound();
15          System.out.println("Cat: Meow!");
16      }
17  }
```

Final в классе, не позволит его наследовать:

```
3   final class ImmutableAnimal { }
4
5   class Cat extends ImmutableAnimal { }
```

Возникнет ошибка: cannot inherit from final class.

Свойство “неизменяемости” класса полезно в том случае, если этот класс используется, например, как обертка над данными (Data class), он будет гарантировать уникальность объекта и при изменении его данных нужно каждый раз создавать новый объект и передавать в него данные старого класса. Это улучшает безопасность и читаемость кода. Это свойство, например, использует класс String из стандартной библиотеки Java.

Статические элементы также невозможно наследовать в дочерних классах, т.к. они привязаны только к тому классу, где они были созданы.

```
@Override
publi  Static methods cannot be annotated with @Override
```

Абстрактный класс

Абстрактный класс в Java — это класс, который нельзя инстанцировать (создать объект), но можно наследовать. Он служит шаблоном для подклассов, объединяя общую логику и определяя абстрактные методы, которые должны быть реализованы в дочерних классах. Его можно создать с помощью ключевого слова **abstract**. Также отмечаются методы, который не должны иметь реализацию и должны переопределяться:

```
3   abstract class Animal {
4       abstract void makeSound();
5   }
6
7   class Cat extends Animal {
8
9       @Override
10      protected void makeSound() {
11          System.out.println("Cat: Meow!");
12      }
13  }
```

Он также может иметь обычные методы, поля и конструкторы:

```java
abstract class Animal {

    private String name;
    abstract void makeSound(); // Абстрактный метод

    public Animal(String name) {
        this.name = name;
    }

    public Animal() {}

    public void sleep() { // Обычный метод
        System.out.println(name + " спит.");
    }
}
```

Абстрактные классы могут иметь конструкторы, которые вызываются при создании объектов подклассов.

Когда использовать абстрактные классы?

- Когда нужно объединить общую логику для группы классов.
- Когда требуется обязать подклассы реализовать специфичные методы.
- Для создания шаблонных методов (часть алгоритма реализована, часть — делегирована подклассам).

Задание

Проект выбирать: номер в списке группы mod 3

11 mod 3 = 2

Нужно скопировать код из репозитория (ссылка ниже) по своему номеру задания, посмотреть, потренироваться и на паре реализовать эти методы при преподавателе, а также одно секретное задание того же типа до 10 минут на всё, без использования любых подсказок, в частности: записи в тетради или листочках или электронном блокноте, мобильный телефон, GPT-модели, Интернет. Возможны вопросы или внесение дополнительных изменений в код.

Задание 0. Зоопарк
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var0

Задание 1. Банк
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var1

Задание 2. Арена
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var3